

CMPU2032 - Programming & Algorithms 2 (2025/26)

Problem Based Assessment

Release Date: 16 April 2026

Submission Date: 3 May 2026 @ 23:59

Overview

Assignment Description

A hospital wants to digitize its patient handling system. You are hired to develop a Python-based system that efficiently manages patient records, emergency cases, and data processing using appropriate data structures and algorithms.

Problem Statement

Design and implement a menu-driven Python program that demonstrates the use of following requirements:

Table Of Contents

1. [Functional Requirements](#)
 - [Task 1: Data Structures Implementation](#)
 - [Task 2: Sorting Algorithms](#)
 - [Menu Interface](#)
 - [Task 3: Testing](#)
 - [Task 4: Design Pattern](#)
2. [Assignment Deliverables](#)

Functional Requirements

Task 1: Data Structures Implementation

1. Linked List (Main Records)

- Store patient records using a linked list.
- Each node contains: Patient ID, Name, Age and Condition.
- Operations: Add Patient, View Patients, Delete Patient and Search Patient.

2. Queue (Emergency Patients)

- Use a queue for emergency cases (FIFO)
 - Operations: Add emergency patient, Process next emergency patient and Display queue.

Task 2: Sorting Algorithms

1. **Merge Sort:** Sort patient records by Patient ID.
2. **Shell Sort:** Sort patient records by Age.

Menu Interface

```
===== Smart Hospital System =====  
1. Add Patient  
2. View Patients  
3. Delete Patient  
4. Search Patient  
5. Add Emergency Patient  
6. Process next Emergency Patient  
7. View Emergency Queue  
8. Sort by ID (Merge Sort)  
9. Sort by Age (Shell Sort)  
10. Exit
```

Enter your choice:

Task 3: Testing

Students must design and include at least 5 test cases, covering: -

- Normal cases (expected valid inputs)
- Edge cases (boundary or unusual inputs)

Task 4: Design Pattern

Suggest at least two design pattern that can be implemented in your application and why the selected ones will be best option?

Assignment Deliverables

1. Submit a ZIP file containing source code.
2. A report with separate task-wise sections describing the implementation, including the following: -
 - Explanation of the data structures used.
 - Description of sorting algorithms implemented.
 - Test cases.
 - Explanation of the design pattern selected.
 - Sample outputs of the program.
3. A short video (maximum 8 minutes) demonstrating and explaining your Python code, including execution of the menu-driven program. The URL of this video must be included in your report.